

Security Assessment Report

Generated: 2026-05-05 11:23

Report ID: 380c1505

Security Assessment Report

Target: /workspace/uploads/39e48c32d11b

Primary Component Identified: server.py

Assessment Basis: Automated static analysis results from Bandit and SonarQubestyle scanning

1. Executive Summary

The automated security review of the provided codebase identified a small number of findings, all concentrated in server.py. No Bandit security issues were reported, and no dependency analysis could be completed because no requirements.txt file was present. Sonar analysis identified several code quality findings, primarily related to excessive function complexity and commentedout code.

From a security perspective, the most important observations are the three Critical findings reported for cognitive complexity. While these are not direct vulnerabilities in the traditional sense, they represent a maintainability and reliability risk that can materially increase the likelihood of future security defects, regression errors, and missed validation logic in complex functions. Two additional Major codesmell findings indicate commentedout code left in the source, which can reduce code clarity and operational assurance.

Overall security risk level: Medium

This rating is driven by the absence of direct exploitable issues in the available scan results, balanced against the presence of highly complex code paths that increase longterm security exposure and maintenance risk.

2. Risk Overview

Findings by Severity

Critical: 3

High: 0

Medium: 0

Low: 0

Informational: 2

Risk Interpretation

Critical: These findings indicate highly complex functions that are difficult to review, test, and maintain. In security-sensitive code, such complexity can conceal logic flaws, missing checks, or unsafe handling of input.

High: No high-severity issues were reported by the scanners.

Medium: No medium-severity issues were reported.

Low: No low-severity issues were reported.

Informational: These findings do not represent direct vulnerabilities, but they should be cleaned up to improve readability, maintainability, and code hygiene.

3. Detailed Findings

Critical Vulnerabilities

Finding 1 — Excessive Cognitive Complexity in server.py

Severity: Critical

CVSS Score (estimated): 8.1

OWASP Category:

A04: Insecure Design

A10: Server-Side Request Forgery / Other — not directly applicable

Primary mapping: A04: Insecure Design

Affected Component:

server.py line 120

Description:

SonarQube reported that the function beginning at line 120 has a cognitive complexity of 26, exceeding the recommended threshold of 15.

Excessive complexity makes the function harder to reason about, test, and secure.

In securityrelevant code, complex branching and nesting can hide missing authorization checks, incomplete validation, or incorrect error handling.

Impact:

Attackers may benefit indirectly if complex logic causes validation or controlflow mistakes.

Business impact includes increased likelihood of defects, slower incident response, and higher risk of introducing vulnerabilities during future code changes.

Proof of Concept (if possible):

No direct exploit is demonstrated by the scan results.

The issue is exploitable only indirectly through logic flaws introduced or overlooked due to the function's complexity.

Recommended Remediation:

Refactor the function into smaller, singlepurpose helper functions.

Reduce nested conditionals and repeated logic.

Add unit tests for each branch and edge case.

Review the function for securitycritical checks such as authentication, authorization, and input validation.

Apply secure coding review before deployment changes.

Finding 2 — Excessive Cognitive Complexity in server.py

Severity: Critical

CVSS Score (estimated): 7.8

OWASP Category:

A04: Insecure Design

Affected Component:

server.py line 151

Description:

SonarQube reported a function at line 151 with cognitive complexity 17, above the allowed threshold.

The function contains multiple conditional branches and nested logic paths, which increases the chance of maintenance errors and overlooked security controls.

Impact:

Complex code can conceal incorrect assumptions and flawed logic.

Security impact may include improper handling of edge cases, inconsistent enforcement of rules, or accidental bypass of protective logic.

Operationally, this increases development cost and regression risk.

Proof of Concept (if possible):

No direct exploit identified.

Risk exists primarily through latent logic defects introduced by complexity.

Recommended Remediation:

Break the function into smaller methods with clear responsibilities.

Flatten nested logic where possible using guard clauses.

Ensure comprehensive test coverage for all execution paths.

Perform a security review to verify that all trust boundaries are handled correctly.

Finding 3 — Excessive Cognitive Complexity in server.py

Severity: Critical

CVSS Score (estimated): 7.9

OWASP Category:

A04: Insecure Design

Affected Component:

server.py line 198

Description:

SonarQube reported a function at line 198 with cognitive complexity 25, significantly above the acceptable threshold.

The reported flow indicates multiple branches and nested conditions across a large block of logic, which makes correctness and security assurance more difficult.

Impact:

Increased probability of implementation mistakes in a security-sensitive function.

Possible business impact includes service instability, inconsistent policy enforcement, and higher likelihood of vulnerable future modifications.

Complex logic is particularly risky in request handling, access control, or data processing workflows.

Proof of Concept (if possible):

No direct exploit can be concluded from static analysis alone.

The concern is the elevated risk of hidden logic flaws rather than an immediately exploitable condition.

Recommended Remediation:

Refactor into smaller, testable units.

Introduce explicit validation and errorhandling layers.

Use coding standards that limit function size and branching depth.

Conduct a targeted manual review of this code path for securitysensitive decisions.

High Vulnerabilities

No highseverity vulnerabilities were identified by the automated scans.

Medium Vulnerabilities

No mediumseverity vulnerabilities were identified by the automated scans.

Low Vulnerabilities

No lowseverity vulnerabilities were identified by the automated scans.

Informational Findings

Finding 4 — CommentedOut Code Present in server.py

Severity: Informational

CVSS Score (estimated): 0.0

OWASP Category:

A04: Insecure Design

A05: Security Misconfiguration

Primary mapping: A05: Security Misconfiguration

Affected Component:

server.py line 26

Description:

SonarQube flagged commentedout code that should be removed.

While not a vulnerability by itself, commentedout code can create confusion, reveal implementation history, or leave outdated logic in the codebase.

In securitysensitive systems, stale code fragments can mislead maintainers and reviewers.

Impact:

Minimal immediate security impact.

Indirectly, it may increase the chance of incorrect assumptions during maintenance and code review.

Can reduce code clarity and professional hygiene.

Proof of Concept (if possible):

Not applicable.

Recommended Remediation:

Remove obsolete commentedout code.

Preserve legitimate design notes in documentation or commit history instead.

Enforce code review standards to prevent commentedout production code from being merged.

Finding 5 — CommentedOut Code Present in server.py

Severity: Informational

CVSS Score (estimated): 0.0

OWASP Category:

A05: Security Misconfiguration

Affected Component:

server.py line 34

Description:

Another instance of commentedout code was identified and should be removed.

This indicates residual development artifacts in the codebase, which can reduce maintainability and readability.

Impact:

No direct exploitation path is evident.

Longterm risk includes confusion during troubleshooting or maintenance and the potential reintroduction of deprecated logic.

Proof of Concept (if possible):

Not applicable.

Recommended Remediation:

Delete unused commentedout code.

Replace with proper version control history and documentation when necessary.

Apply automated linting rules to detect and block commentedout code in future commits.

4. Recommendations (Global)

1. Refactor complex functions

Split large functions into smaller units with single responsibilities.

Reduce branching and nesting depth.

Add unit tests for each path and edge case.

2. Prioritize secure design and review

Review all complex requesthandling or decisionmaking logic for missing authorization, validation, and error checks.

Perform manual security review on the functions flagged as highly complex.

3. Improve code hygiene

Remove commentedout code and other stale artifacts.

Enforce linting and static analysis in CI/CD pipelines.

4. Strengthen dependency governance

Add and maintain a requirements.txt or equivalent dependency manifest.

Perform dependency scanning and version pinning.

Keep thirdparty packages updated and monitor for known vulnerabilities.

5. Validate inputs and outputs

Ensure all usercontrolled inputs are validated, normalized, and safely handled.

Apply output encoding where relevant to prevent downstream security issues.

6. Use securityfocused development standards

Adopt secure coding guidelines aligned with OWASP Top 10.

Require peer review for any function that handles authentication, authorization, file access, or external requests.

7. Automate quality and security gates

Keep Bandit, Sonar, and dependency scanning in CI.

Fail builds on critical security or design findings where appropriate.

5. Conclusion

The scan results do not show direct exploitable vulnerabilities from Bandit, but they do reveal several maintainability and design concerns in server.py, including three critical cognitive complexity findings. These issues are not immediate exploits, but they materially increase the likelihood of future security defects and make the code harder to trust in production.

The most urgent action is to refactor the affected functions and review them for any hidden security logic weaknesses. Although the current overall risk is assessed as Medium, addressing the critical complexity findings should be treated as a priority to reduce future exposure and improve the robustness of the application.